

Estándares de programación

Fecha de Creación:	May 22, 2025
Ultima modificación:	May 22, 2025
Creado por:	@Hernan Caballero
Versión:	1.0

Estos estándares están sujetos a cambios constantes, en mejora de la aplicación.
De la misma manera se iran agregando tips de programación.

ESTRUCTURAS (ORACLE)

Todos los scripts de estructura deben llevar el owner obligatoriamente en cada sentencia.

Procedures and Functions.

1. No se deben crear Procedures o Functions sueltos, todos los procedimientos y funciones deben ir en Packages.

Packages

1. El nombre del package tiene la siguiente estructura: SK<módulo>_<función>
<módulo> es el módulo al que pertenece el package (FA,IN,CC,CP,IT,CG,CO,IM,etc)
<función> palabra que indica la funcionalidad del módulo. Hay packages que no necesitan la función (SKWM,SKIT,SKCK) dado que el módulo no es tan complejo como para requerir múltiples packages.
2. Los comentarios de cada función o procedimiento van en el encabezado del package.
3. Los nombres de las funciones y procedimientos deben ser lo más sencillos y representativos posibles.
4. Todo procedimiento debe tener una variable P_ERROR inicial de salida que debe retornar el error que ocurrió dentro del procedimiento.
5. En lo posible el procedimiento debe hacer rollback de los cambios que originó, con la excepción de los procedimientos que se llaman desde triggers.

Tablas

1. El nombre de la tabla tiene la siguiente estructura:
<tipo><módulo><función>

<tipo> AR (tabla normal) HI (Historica) TM (temporal)

<módulo> el módulo principal al que pertenece la tabla (FA,IN,CC,CP,IT,CG,CO,IM,etc)

<función> 2 o 3 letras que indiquen la función de la tabla. Se establece un nuevo criterio donde 2 letras indican la tabla principal y la tercera indica una tabla satélite. (pe. ARITSO es la solicitud de soporte y ARITSOT es el tipo de solicitud de soporte).

2. Se debe reducir al mínimo los campos que puedan contener valores nulos. Esto evita que la codificación se llene de la función nvl. Por lo general los campos llaves, referencias, tipo, estado, descripción no deben ser nulos, de la misma manera los campos monto, precio, costo, etc.
3. No se debe agregar cláusulas de storage a las sentencias de creación.
4. Los constraints de las tablas deben ir en sentencias separadas del create table.
5. Se debe crear un sinónimo para la tabla.
6. Se da grant de select, update, delete, insert a PUBLIC. A futuro debemos cambiar esta práctica, cuando se tenga un manejo de roles funcional.
7. Se deben crear los foreign keys de la tabla y debe existir un índice para cada foreign key. El nombre de los foreign key debe ser <tabla><tabla referenciada> y el índice <tabla><tabla referenciada>_fk.
8. Se debe crear el primary key de la tabla. El nombre del primary key debe ser <tabla>_pk. La sentencia que crea el primary key siempre debe llevar la opción USING INDEX.
9. La tabla debe llevar un comentario que no deje duda del uso de la tabla. Y las columnas con una funcionalidad específica deben llevar el comentario obligatorio. Las excepciones al comentario de la columna son campos como NO_CIA, CENTRO, ESTADO.
10. Toda tabla con múltiples estados y tipos debe tener por obligación la tabla satélite con la descripción de estos datos.
11. El script de creación de la tabla puede contener inserción de los registros iniciales de la tabla.
12. Toda tabla debe tener los campos FECHA_CREA, USUARIO_CREA, FECHA_MODIFICA y USUARIO_MODIFICA como auditoría mínima.
13. Todo índice adicional (performance y búsquedas) debe nombrarse <tabla>_<campo principal del índice>_idx
14. Los **triggers** llevan la nomenclatura <tabla>_<momento><acción><fila> donde
<momento> es B para Before o A para after
<acción> es I para Insert, U para Update, D para Delete, si se aplica el trigger en más de una función se omite.
<fila> es R para EACH ROW, de lo contrario se omite.

Se debe tener la **menor cantidad de triggers posible** sobre una tabla. Hay excepciones como los triggers de WMS que por modularidad se separan de los triggers propios del ERP.

15. Toda tabla nueva debe tener un trigger como el que sigue como auditoría mínima:

```
1 CREATE OR REPLACE
2 TRIGGER it53.aritafm_br
3 BEFORE INSERT OR UPDATE
4 ON it53.aritafm
5   REFERENCING NEW AS NEW
6               OLD AS OLD
7   FOR EACH ROW
8 BEGIN
9   IF INSERTING THEN
10    :NEW.usuario_crea := USER;
11    :NEW.fecha_crea   := SYSDATE;
12  ELSIF UPDATING THEN
13    :NEW.usuario_modifica := USER;
14    :NEW.fecha_modifica  := SYSDATE;
15  END IF;
16 END;
```

Otros

1. El nombre de la **vista** tiene la siguiente estructura:

<tipo><módulo><función>

<tipo> VW (vista normal)

<módulo> el módulo principal al que pertenece la vista (FA,IN,CC,CP,IT,CG,CO,IM,etc)

<función> 2 o 3 letras que indiquen la función de la vista.

2. Se debe crear el sinónimo de la vista y dar el grant de select a public.

3. El nombre de las **secuencias** tiene la siguiente estructura:

<tipo><módulo><función>

<tipo> SQ (secuencia normal)

<módulo> el módulo principal al que pertenece la secuencia(FA,IN,CC,CP,IT,CG,CO,IM,etc)

<función> 2 o 3 letras que indiquen la función de la secuencia. solicitud de soporte).

4. Se debe crear el sinónimo de la secuencia y dar el grant de select a public.

PL/SQL (ORACLE)

La velocidad con que se varían los procesos en nuestra empresa obliga a que hagamos todo lo posible porque el mantenimiento y modificación de los programas sea lo más rápida.

1. **La indentación no es opcional.** Es una necesidad para el entendimiento rápido de los programas. De preferencia evitar las tabulaciones, no todos los editores las entienden igual.

2. **Manejo de errores**, se ha estado implementando basados en múltiples fuentes el uso del v_step para el manejo de errores, esto permite evitar “traps” de errores por anidamiento entre otras.

3. Es preferible que todo procedimiento lleve un savepoint y rollback. Si embargo esto aun depende de la implementación, hay muchos casos en que este esquema no se puede usar debido a que por ejemplo los procedimientos son llamados desde triggers.
4. Los comentarios deben ser concisos. Los comentarios “pesados” que se requieran para el entendimiento de la función del programa deben ir en el encabezado del package.
5. No abusar de los **BEGIN y END**, es una practica muy comun abusar de ponerle BEGIN y END a los bloques PL/SQL al final tienden a confundir a quien lee el código. El BEGIN y END sólo deben utilizarse cuando se quiere declarar variables (van acompañados del DECLARE) o cuando se quiere dar un manejo de errores (van acompañados del EXCEPTION); en este último caso sólo para manejos especiales ya que la tendencia es usar lo que indica el punto 2.
6. Evitar el uso de **WHEN OTHERS**. El uso del WHEN OTHERS tiende a enmascarar los errores. Si es necesario utilizar el WHEN OTHERS siempre debe ser para emitir algún tipo de mensaje de error.
7. **Los cursores se definen en línea**, no se usa FETCH, ni se definen en el DECLARE. Esto hace más legible el código.

Catalogo de modulos

Modulo	Descripción
bee	Beetrack
ci	Maestro de Clientes
fa	Facturación
hi	Históricos
in	Maestro de Artículos
it	IT
ms	Integraciones
sx	Sincronizaciones
tl	Etl's